

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Applied Sciences · Department of Computer Science

Solution Book 8

SCS 2107 Structured Programming, January 2013 (C++)

SCS 1101 Introduction to Computer Science and Programming

Past-paper solutions with marks allocated, algorithms, textbook flowcharts, exam-ready code and a civil & water engineering lens

Second-year Structured Programming paper, answered in C++ · any 4 of 6 · 25 marks each

Book 8 of 8 · Prepared October 2026

Index — tap a tab or a line to jump

Every chapter starts with the coloured tab bar below; tap any tab to jump straight to that question. The page footer has a link back to this index. In Word, the Navigation Pane shows the same chapters.

Guide **How to use this book**

Glance **At a glance**

Q1 **Question 1 [25]**

- ▸ 1(a) The structured programming constructs, with diagrams [12]
- ▸ 1(b) What is a structured walk-through? [3]
- ▸ 1(c) Advantages and disadvantages of structured walk-throughs [10]

Q2 **Question 2 [25]**

- ▸ 2(a)(i) What is an infinite loop, and how can it be useful? [5]
- ▸ 2(a)(ii) What is wrong with this loop? [5]
- ▸ 2(b) Convert the for loop into a while loop [7]
- ▸ 2(c) Simple calculator with a switch statement [8]

Q3 **Question 3 [25]**

- ▸ 3(a) Function print_pyramid(height) [14]
- ▸ 3(b) Recursive Fibonacci function [11]

Q4 **Question 4 [25]**

- ▸ 4(a) Explain [9]
- ▸ 4(b) Number of the last element of int test[5]; [2]
- ▸ 4(c) Output of the code in Figure 3 [5]
- ▸ 4(d) Function reverse(array, n) [9]

Q5 **Question 5 [25]**

- ▸ 5(a) Explain [3 each = 12]
- ▸ 5(b) Program: NUST mark sheet with a base class and a derived class [13]

Q6 **Question 6 [25]**

- ▸ Compare and contrast structured programming and object-oriented programming

Refs **Sources for this book**

Links **Source register**

How to use this book

This book is one of eight stand-alone solution books for NUST SCS 1101 past papers. It solves **SCS 2107 Structured Programming, January 2013 (C++)** question by question, with the marks the examiner allocates shown in square brackets, e.g. [2].

The six-block answer template

Every answer in this book (and in all eight books of the set) is built from the same six blocks, in the same order. Theory answers use blocks 1, 2, 5 and 6; programming answers use all six.

1. **Like you're 3** — one or two sentences with a toy, a bath or a water tap.
2. **Model answer** — the exam wording, with the mark for each point in brackets, e.g. [2]. Write it like this and you collect every mark.
3. **Algorithm + code** — numbered pseudocode first, then C code that follows the **H-D-I-P-O-R** recipe.
4. **Flowchart or structure chart** — textbook (ANSI) symbols. Programs built from functions also get a structure (hierarchy) chart.
5. **Engineering lens** — the same idea in a civil and water engineering setting, using the profession's vocabulary.
6. **Examiner's tip** — where students usually lose marks.

The H-D-I-P-O-R code recipe — "**H**appy **D**ucks **I**n **P**onds **O**nly **R**elax". Every exam program is these six steps, in this order:

```c
/* H - Header: what the program does, and the libraries */
#include <stdio.h>

int main(void)
{
    /* D - Declare: every variable, with a sensible type and a starting value */
    /* I - Input:   printf a prompt, then scanf with & */
    /* P - Process: the if-ladder, loop or formula */
    /* O - Output:  printf the result with units and decimals */
    /* R - Return:  */ return 0;
}
```

The recipe is worth marks by itself: examiners give separate marks for headers, declarations, input, processing and output. A half-finished program laid out this way still scores; a clever one-liner without structure often does not.

Algorithm style used in every book:

```
Step 1: Start
Step 2: Read <inputs>
Step 3: <process, one action per step>
Step 4: Display <outputs>
Step 5: Stop
```

Flowchart symbols (ANSI):

Symbol	Shape	Use	Colour in this book
Terminator	Rounded box (stadium)	Start / Stop	Green

Symbol	Shape	Use	Colour in this book
Input/Output	Parallelogram	Read or display	Blue
Process	Rectangle	Calculation or assignment	Amber
Decision	Diamond	Yes/no test, two exits	Red
Predefined process	Rectangle with double sides	Call to a function	Purple
Flow line	Arrow	Order of steps	Dark grey arrow

The smallest possible program, drawn with these symbols:

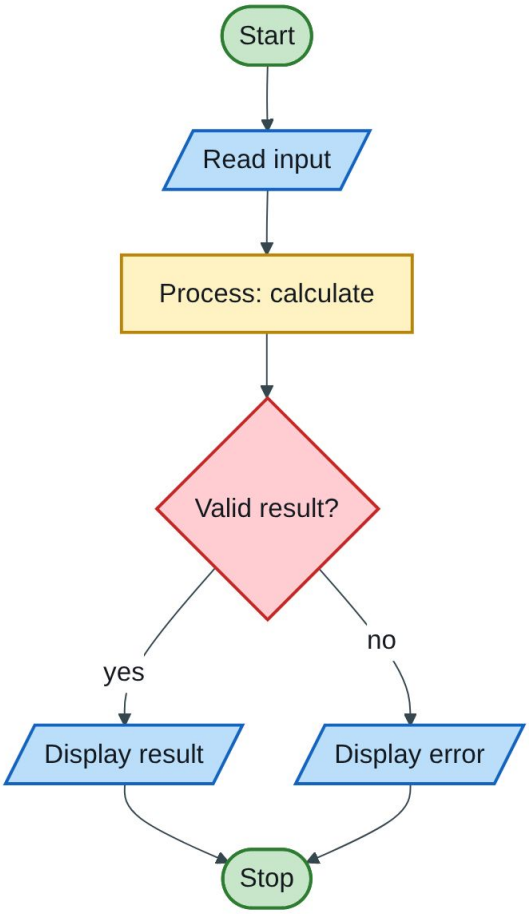


Figure 8.1 — Flowchart

The engineering lens. Programming questions get a civil and water engineering version where it fits, for example:

- tariff ladders become **increasing block tariffs** for water;
- array statistics become **rainfall** or **concrete cube strength** analysis;
- geometry programs become **cross-section, wetted perimeter and hydraulic radius** calculations.

Write the exam version in the exam; the engineering version is for understanding and for your later modules.

How the solutions were researched

Every fact a solution relies on is traced to a source in one of four tiers; each source has a short key (e.g. [MIT-2]) so answers can cite it without long links.

1. **Tier 1 — the course itself.** The SCS 1101 outline and the past papers decide *what* to answer and how marks are split.

2. **Tier 2 — university teaching material.** MIT OpenCourseWare and UCLA course pages for programming and algorithms; university histories for milestones.
3. **Tier 3 — standards and research.** NIST for cloud definitions; peer-reviewed and arXiv papers, including Wits research on cloud adoption in South Africa.
4. **Tier 4 — standard textbook knowledge.** Definitions every textbook agrees on (OS functions, topologies, printers) are written from the textbooks your outline lists, without a web citation.

A claim is only cited when a source was actually opened; anything from tier 4 stays general and uncited.

At a glance

SCS 2107 Structured Programming, January 2013. 3 hours, six questions, **answer any four**, 25 marks each, **all code in C++**. C++ is C plus classes: `cin`/`cout` replace `scanf`/`printf`, and the same H-D-I-P-O-R recipe applies.

Question	What it tests	Pick?
Q1	Structured programming constructs with diagrams; structured walk-through	Yes, pure theory
Q2	Infinite loops; loop bug; for → while; switch calculator	Yes, short code
Q3	`print_pyramid` function; recursive Fibonacci	Yes
Q4	Arrays explained; code output; `reverse` function	Yes
Q5	Constructors, inline, virtual, inheritance; mark-sheet classes	If OOP is strong
Q6	Structured programming vs OOP (25-mark essay)	Backup

C ↔ C++ quick map: `#include <iostream>` + `using namespace std;`; `cout << x;` prints; `cin >> x;` reads (no `&`); `endl` is a new line; `string` holds text; `class` bundles data and functions.

Question 1 [25]

1(a) The structured programming constructs, with diagrams [12]

Like you're 3: every program is made from three Lego shapes only: **straight road** (sequence), **fork in the road** (selection) and **roundabout** (iteration).

For each construct: diagram [2], explanation [1], C++ snippet [1].

1. Sequence — statements execute one after another in the order written.

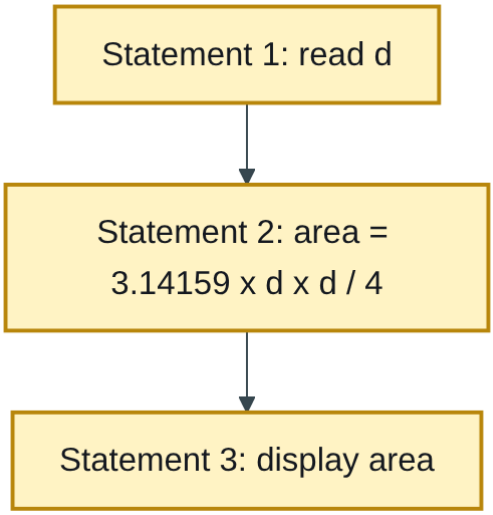


Figure 8.2 — Flowchart

```
cin >> d;
area = 3.14159 * d * d / 4;
cout << "Area = " << area << endl;
```

2. Selection (decision) — a condition decides which of two or more paths is taken (`if`, `if-else`, `switch`).

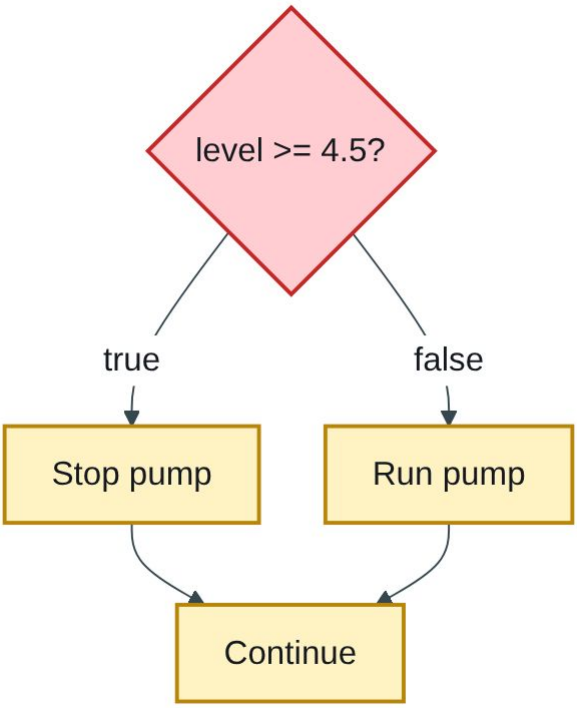


Figure 8.3 — Flowchart

```
if (level >= 4.5) cout << "Stop pump";
else              cout << "Run pump";
```

3. Iteration (repetition) — a block repeats while a condition is true (`while`, `do-while`, `for`).

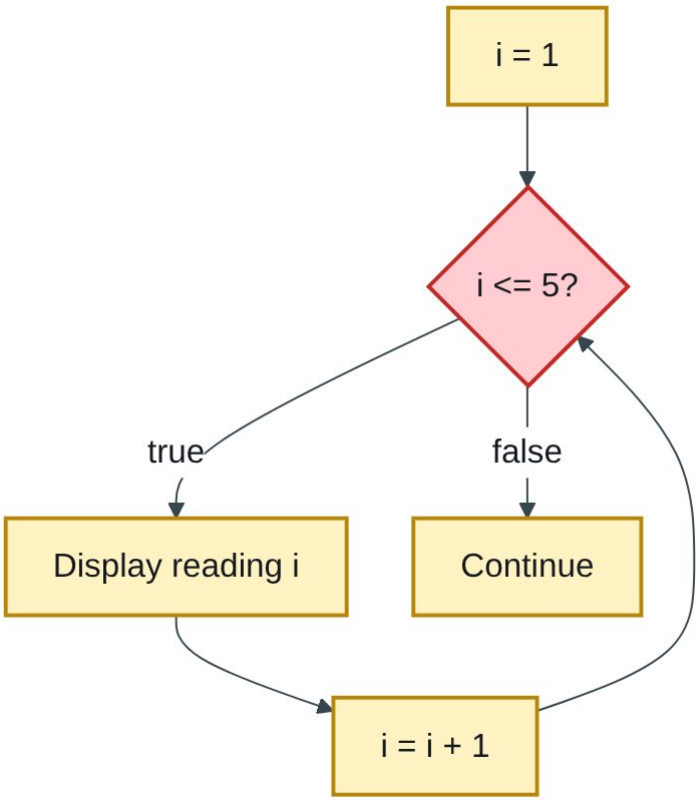


Figure 8.4 — Flowchart

```
for (int i = 1; i <= 5; i++)
    cout << "Reading " << i << endl;
```

Each construct has **one entry and one exit**, so constructs can be nested inside each other without `goto` [this sentence earns the remaining marks for understanding].

1(b) What is a structured walk-through? [3]

Like you're 3: before the school play, you act it out for your friends so they can spot mistakes before the real show.

A structured walk-through is a **planned peer-review meeting** in which the programmer leads a small team (reviewers, a moderator and a recorder) **step by step through the design or code** [1], explaining its logic so that errors, omissions and departures from standards are found **early, before testing** [1]. Problems are recorded for the author to fix; the code is not corrected in the meeting itself [1].

1(c) Advantages and disadvantages of structured walk-throughs [10]

Advantages (any five, 1 each)	Disadvantages (any five, 1 each)
Errors are found early, when they are cheapest to fix	Time-consuming for several people
Improves program quality and readability	Needs preparation: copies, agenda, reviewers' study time
Enforces coding standards across the team	Can feel like personal criticism and cause tension
Spreads knowledge: others understand the code	Quality depends on the reviewers' skill and attention
Trains junior programmers	May miss run-time and performance errors that only testing reveals
Shared ownership: the team, not one person, is responsible	Scheduling meetings delays the project

Engineering lens: engineering offices do the same thing as a **design review**: a checking engineer walks through calculations and drawings before they are issued for construction, because a mistake caught on paper costs far less than one found on site.

Question 2 [25]

2(a)(i) What is an infinite loop, and how can it be useful? [5]

Like you're 3: a merry-go-round that never stops on its own; sometimes that is exactly what you want, as long as there is an off switch.

An **infinite loop** is a loop whose condition never becomes false, so it runs forever unless stopped from inside (e.g. `break`, `return`) or from outside [2]. Useful cases [3]:

- **Servers and operating systems** — wait forever for requests or events.
- **Embedded and control systems** — a pump controller or PLC reads sensors and updates outputs in an endless scan loop.
- **Menus and games** — repeat until the user chooses "exit", which triggers `break`.

```
while (true) {                  // deliberate infinite loop
    int choice = showMenu();
    if (choice == 4) break;     // the planned way out
    handle(choice);
}
```

2(a)(ii) What is wrong with this loop? [5]

```
while (n <= 100)
    sum += n*n;
```

Like you're 3: the robot keeps adding blocks but never takes a step forward, so it stands in the same spot forever.

1. **n never changes** inside the loop, so `n <= 100` stays true: an **infinite loop** [2].
2. Without braces, only the one statement belongs to the loop, so an increment added on the next line would sit **outside** it [1].
3. `sum` and `n` must be **initialised** before the loop, or the result is garbage [1].

Corrected [1]:

```
int n = 1, sum = 0;
while (n <= 100) {
    sum += n * n;
    n++;                    // move towards the exit condition
}
```

2(b) Convert the for loop into a while loop [7]

```
for (int i = 1; i <= n; i++)
    cout << i*i << " ";
```

Like you're 3: a `for` loop is a lunchbox with three compartments (start; test; step). A `while` loop is the same food unpacked: start before, test at the top, step at the bottom.

```
int i = 1;                  // initialise   (before the loop)   [2]
while (i <= n) {            // test          (in the brackets)   [2]
```

```
    cout << i*i << " ";    // body                                [1]
    i++;                   // update        (end of the body)     [2]
}
```

For n = 5 both print `1 4 9 16 25`.

2(c) Simple calculator with a switch statement [8]

Like you're 3: you give the robot two numbers and a sign; it looks up the sign in its list and does that sum.

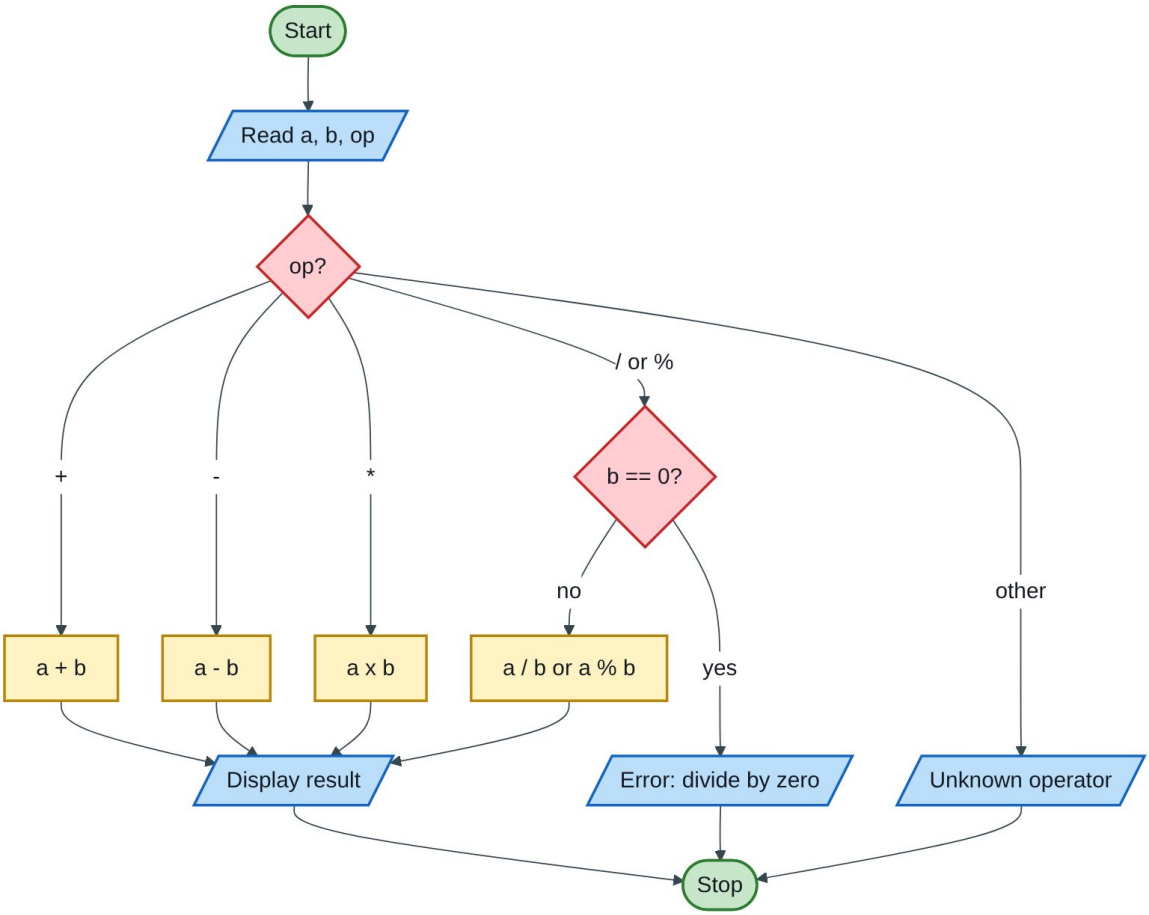


Figure 8.5 — Flowchart

```
// H: simple calculator - two integers and an operator, using switch
#include <iostream>
using namespace std;

int main()
{
    int a, b;                                          // D
    char op;

    cout << "Enter two integers and an operator (+ - * / %): ";   // I
    cin >> a >> b >> op;

    switch (op) {                                      // P + O
    case '+': cout << a << " + " << b << " = " << a + b << endl; break;
    case '-': cout << a << " - " << b << " = " << a - b << endl; break;
    case '*': cout << a << " * " << b << " = " << a * b << endl; break;
    case '/':
        if (b != 0) cout << a << " / " << b << " = " << a / b << endl;
```

```
        else        cout << "Error: division by zero" << endl;
        break;
    case '%':
        if (b != 0) cout << a << " % " << b << " = " << a % b << endl;
        else        cout << "Error: division by zero" << endl;
        break;
    default:
        cout << "Unknown operator: " << op << endl;
    }
    return 0;                                          // R
}
```

Test: `17 5 /` → `17 / 5 = 3` (integer quotient); `17 5 %` → `17 % 5 = 2`; `4 0 /` → error message.

Marking guide [8]: reading two integers and a character [2]; `switch` with all five cases and `break`s [4]; division-by-zero check [1]; `default` case [1].

Question 3 [25]

3(a) Function `print_pyramid(height)` [14]

Like you're 3: each row of the pyramid has 2 more stars than the row above. To make it look like a pyramid instead of a staircase, push each row in with some spaces: more spaces at the top, none at the bottom.

Pattern for height 6 (Figure 2.0): row r has **2r stars** (2, 4, 6, ..., 12) and **height − r leading spaces**.

Row r	Spaces (6 − r)	Stars (2r)
1	5	2
2	4	4
3	3	6
4	2	8
5	1	10
6	0	12

Flowchart:

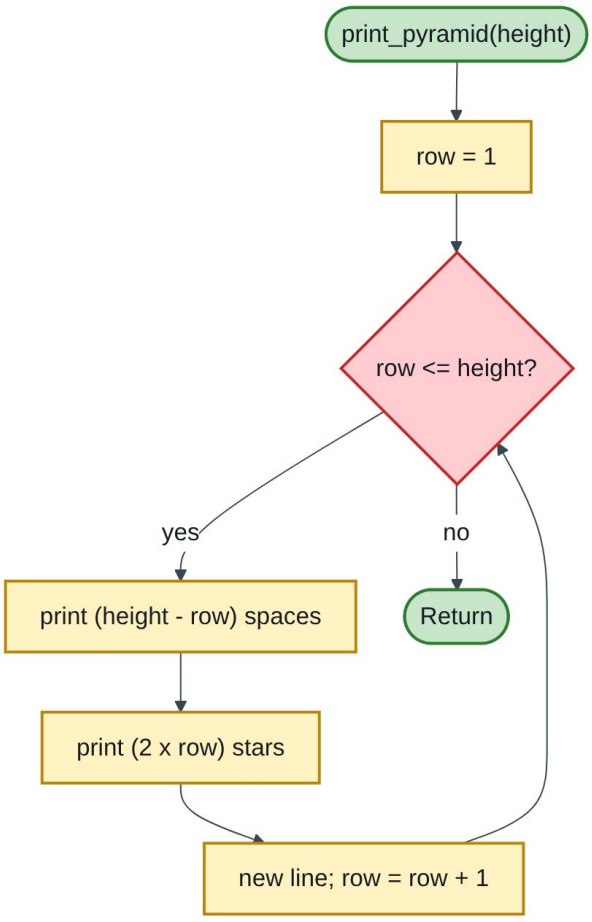


Figure 8.6 — Flowchart

Code:

```cpp
// H: print a pyramid of '*' of the given height (row r has 2r stars)
#include <iostream>
using namespace std;

void print_pyramid(int height)
{
    for (int row = 1; row <= height; row++) {
        for (int s = 0; s < height - row; s++)   // leading spaces centre the row
            cout << ' ';
        for (int k = 0; k < 2 * row; k++)        // 2, 4, 6 ... stars
            cout << '*';
        cout << endl;
    }
}

int main()
{
    int h;
    cout << "Height of pyramid: ";
    cin >> h;
    print_pyramid(h);
    return 0;
}
```

Output for height 6:

```
     **
    ****
   ******
  ********
 **********
************
```

If the printed figure is meant to be left-aligned, simply delete the spaces loop.

Marking guide [14]: function header with an `int height` parameter and `void` return [2]; outer row loop [3]; spaces loop [3]; stars loop using 2 × row [4]; new line after each row [1]; call from `main` [1].

3(b) Recursive Fibonacci function [11]

Like you're 3: each number is the two numbers before it added together: 0, 1, 1, 2, 3, 5, 8… The function asks two smaller copies of itself, until it reaches 0 or 1, which it already knows.

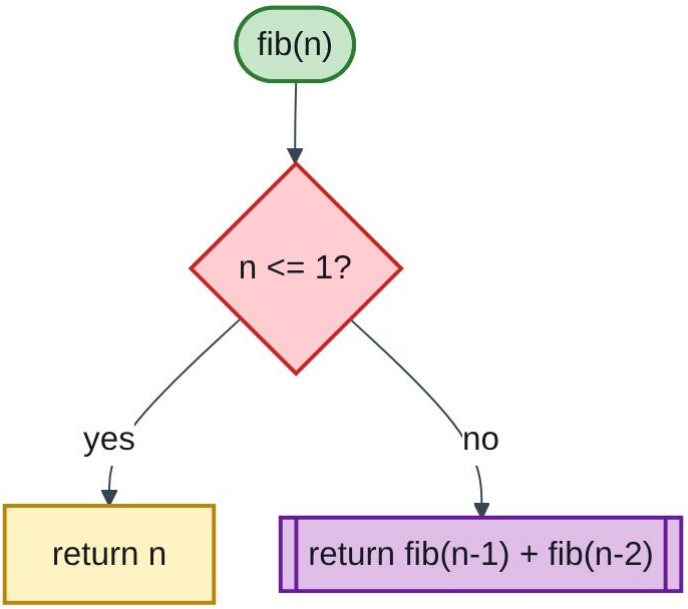


Figure 8.7 — Flowchart

```
// H: Fibonacci series using a recursive function
#include <iostream>
using namespace std;

long fib(int n)
{
    if (n <= 1)                      // base cases: fib(0) = 0, fib(1) = 1
        return n;
    return fib(n - 1) + fib(n - 2);   // recursive case
}

int main()
{
    int terms;
    cout << "How many terms? ";
    cin >> terms;
    for (int i = 0; i < terms; i++)
        cout << fib(i) << " ";
    cout << endl;
    return 0;
}
```

Test: 10 terms → `0 1 1 2 3 5 8 13 21 34`.

Marking guide [11]: function header [2]; base case [3]; recursive case [3]; `main` reading the number of terms and printing the series [2]; output [1].

One line for extra credit: naive recursion recomputes the same values and grows exponentially; storing each result once (memoization) makes it linear (MIT-2).

Question 4 [25]

4(a) Explain [9]

Like you're 3: a simple variable is one egg cup; an array is an egg tray; a 2-D array is a tray with rows and columns; a 3-D array is a stack of trays.

(i) Array vs simple variable [3]: a simple variable holds **one** value under one name [1]; an array holds **many values of the same type** under one name, stored next to each other in memory and reached by an index starting at 0 [1]. Arrays let one loop process hundreds of values that would otherwise need hundreds of separate variables [1].

Declaration	Meaning	Elements
(ii) `double a[5];` [2]	One-dimensional array of 5 doubles, `a[0]` to `a[4]`	5
(iii) `float b[3][5];` [2]	Two-dimensional array: 3 rows × 5 columns of floats, `b[0][0]` to `b[2][4]`	15
(iv) `int c[5][4][10];` [2]	Three-dimensional array: 5 × 4 × 10 ints, `c[0][0][0]` to `c[4][3][9]`	200

4(b) Number of the last element of `int test[5];` [2]

The last element is **`test[4]`**: indexes start at 0, so 5 elements are numbered 0 to 4 [2].

4(c) Output of the code in Figure 3 [5]

As printed, the first line reads `include <iostream>` **without the #**, so the program would not compile. Say so in one line [1], then give the output assuming `#include <iostream>` [4]:

```
Members of the array
Distance 1: 44.14
Distance 2: 720.52
Distance 3: 96.08
Distance 4: 468.78
Distance 5: 6.28
```

The loop runs i = 0 to 4 and prints `i + 1`, so the labels run 1 to 5 while the indexes run 0 to 4.

4(d) Function `reverse(array, n)` [9]

Like you're 3: two children stand at the two ends of the row and swap their cards, then each takes one step inwards and swaps again, until they meet in the middle.

Step	i	j	Swap	Array afterwards
Start	—	—	—	5.8, 2.6, 9.0, 3.4, 7.1
1	0	4	5.8 ↔ 7.1	7.1, 2.6, 9.0, 3.4, 5.8

Step	i	j	Swap	Array afterwards
2	1	3	2.6 ↔ 3.4	7.1, 3.4, 9.0, 2.6, 5.8
3	2	2	stop (i is not < j)	7.1, 3.4, 9.0, 2.6, 5.8

```
// reverses the first n values of a float array in place; returns nothing
void reverse(float a[], int n)
{
    for (int i = 0, j = n - 1; i < j; i++, j--) {
        float temp = a[i];      // three-step swap
        a[i] = a[j];
        a[j] = temp;
    }
}
```

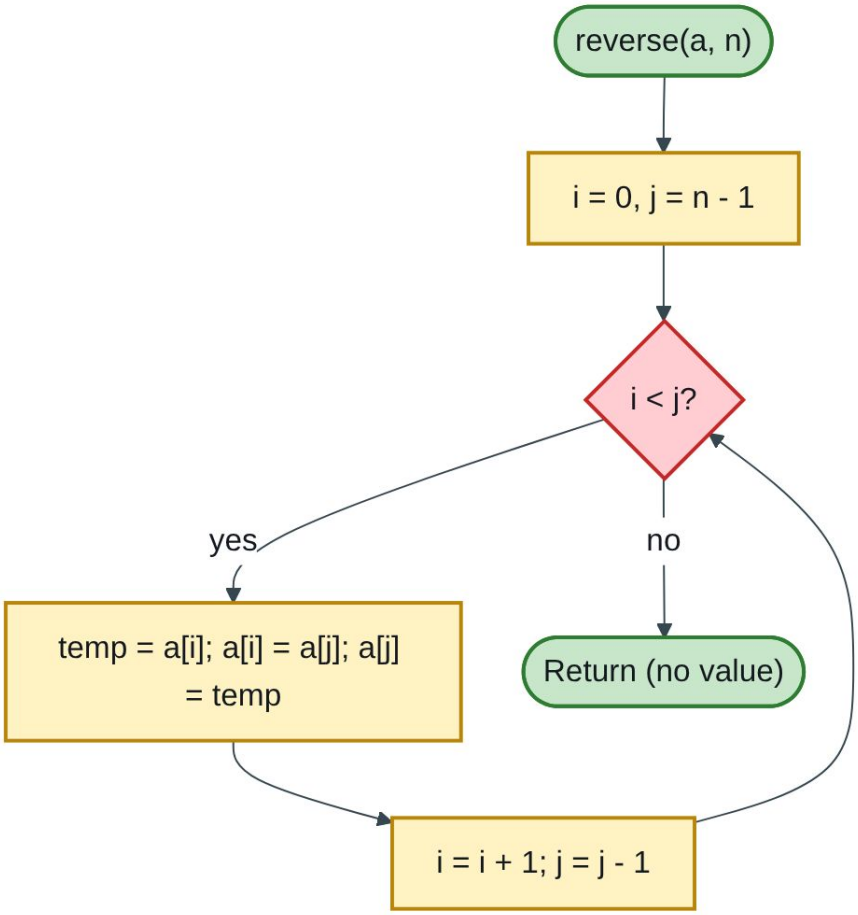


Figure 8.8 — Flowchart

The array is changed for the caller because C++ passes arrays by address.

Marking guide [9]: `void` function with an array and an int parameter [2]; loop moving i up and j down while i < j [4]; correct swap with a temporary variable [2]; no return value [1].

Engineering lens: surveyors reverse ordered lists all the time, e.g. turning a **levelling run** recorded from benchmark A to B into the B-to-A direction, or reversing chainages along a pipeline.

Question 5 [25]

5(a) Explain [3 each = 12]

Like you're 3: a constructor is the factory that builds a toy; a *non-public* one means only the toy company may build it. An inline function is a tiny instruction copied straight into the story instead of saying "see page 50". A virtual function is a button that does different things depending on which toy you press it on. Inheritance is a child toy getting its parent's features.

(i) Non-public constructors — constructors declared `private` or `protected` [1]. A **private** constructor stops code outside the class from creating objects directly; objects are made only through a static member function, e.g. the **Singleton** pattern that allows exactly one object [1]. A **protected** constructor lets only derived classes build the object, so the base class is used purely as a parent [1].

```
class Logger {
private:
    Logger() {}                          // nobody outside can call this
public:
    static Logger& get() { static Logger one; return one; }   // the only Logger
};
```

(ii) Inline function — the `inline` keyword asks the compiler to **replace each call with the function's body** [1], removing call overhead for very small, frequently used functions [1]. Functions defined inside a class body are inline automatically; large functions should not be inline because they bloat the program [1].

```
inline double area(double d) { return 3.14159 * d * d / 4; }
```

(iii) Virtual functions — a member function declared `virtual` in a base class and **overridden** in derived classes [1]. When called through a base-class pointer or reference, the version for the **actual object** runs, decided at run time (**dynamic binding**): this is **run-time polymorphism** [1]. A pure virtual function (`= 0`) makes the class abstract [1].

```
class Section { public: virtual double area() const = 0; };
class Circle : public Section { double d; public: Circle(double x) : d(x) {}
    double area() const override { return 3.14159 * d * d / 4; } };
// Section* s = new Circle(0.3);  s->area();  calls Circle::area at run time
```

(iv) Types of inheritance [3]:

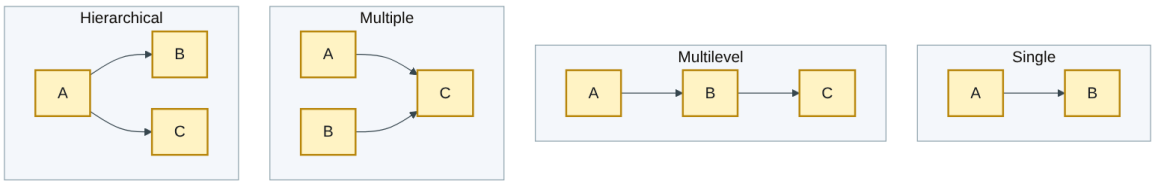


Figure 8.9 — Flowchart

Single (one parent, one child); **multilevel** (a chain A → B → C); **multiple** (one child, several parents); **hierarchical** (one parent, several children); **hybrid** (a mix). Access modes `public`, `protected` and `private` decide how inherited members can be used.

5(b) Program: NUST mark sheet with a base class and a derived class [13]

Like you're 3: the parent card holds who the student is; the child card inherits that and adds which course they took and their marks.

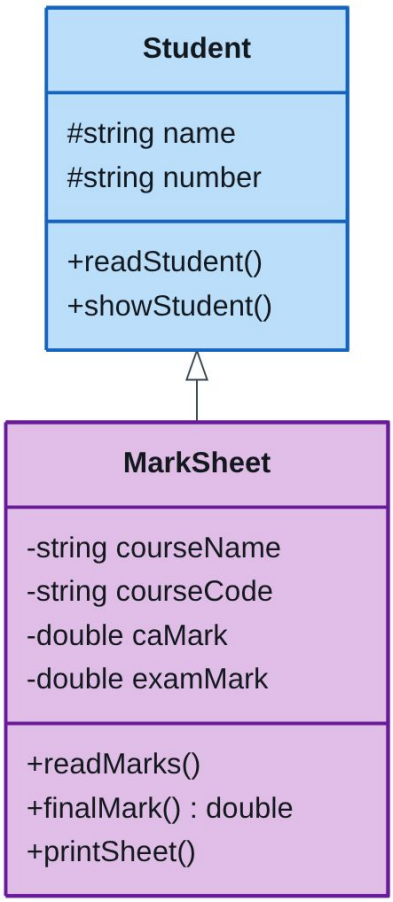


Figure 8.10 — Class diagram

```
// H: NUST examination mark sheet - base class Student, derived class MarkSheet
#include <iostream>
#include <string>
using namespace std;

class Student {                                  // base class
protected:
    string name, number;
public:
    void readStudent() {
        cout << "Student name: ";   getline(cin, name);
        cout << "Student number: "; getline(cin, number);
    }
    void showStudent() const {
        cout << "Name   : " << name   << endl;
        cout << "Number : " << number << endl;
    }
};

class MarkSheet : public Student {               // derived class
private:
    string courseName, courseCode;
    double caMark, examMark;
```

```
public:
    void readMarks() {
        cout << "Course name: "; getline(cin, courseName);
        cout << "Course code: "; getline(cin, courseCode);
        cout << "Continuous assessment mark (/100): "; cin >> caMark;
        cout << "Exam mark (/100): ";                  cin >> examMark;
    }
    double finalMark() const { return 0.4 * caMark + 0.6 * examMark; }
    void printSheet() const {
        cout << "\n====== NUST EXAMINATION MARK SHEET ======\n";
        showStudent();                              // inherited function
        cout << "Course : " << courseName << " (" << courseCode << ")" << endl;
        cout << "CA     : " << caMark   << endl;
        cout << "Exam   : " << examMark << endl;
        cout << "Final  : " << finalMark() << endl;
    }
};

int main()
{
    MarkSheet sheet;
    sheet.readStudent();      // from the base class
    sheet.readMarks();        // from the derived class
    sheet.printSheet();
    return 0;
}
```

The 40% CA / 60% exam weighting follows the current SCS 1101 outline (NUST-1); the question gives no weighting, so state yours.

Test: CA 70, exam 60 → final = 0.4 × 70 + 0.6 × 60 = 28 + 36 = **64**.

Marking guide [13]: base class with name, number and functions [3]; derived class using `public` inheritance with the four extra data members [4]; input functions [2]; formatted mark sheet output [2]; `main` creating the object and calling inherited and own functions [2].

Question 6 [25]

Compare and contrast structured programming and object-oriented programming

Like you're 3: structured programming is a **recipe**: step 1, step 2, step 3, with the ingredients on the side. Object-oriented programming is a **team of toy robots**: each robot carries its own things and knows its own tricks, and you ask the robots to do jobs.

Plan for 25 marks: definitions [4] → similarities [5] → differences [12] → conclusion [4].

Definitions [4]:

- **Structured (procedural) programming** organises a program as a hierarchy of **functions/procedures** built only from sequence, selection and iteration, with data passed between functions. Examples: C, Pascal, FORTRAN.
- **Object-oriented programming (OOP)** organises a program as interacting **objects**, each bundling data (attributes) and the functions that act on it (methods), defined by classes. Examples: C++, Java, Python.

Similarities [5]:

1. Both aim for programs that are easier to read, test and maintain than unstructured "spaghetti" code.
2. Both use sequence, selection and iteration inside their functions or methods.
3. Both break big problems into smaller modules (functions or classes).
4. Both support reuse: function libraries or class libraries.
5. Both can be written in C++, which supports both styles.

Differences [12]:

Point	Structured programming	Object-oriented programming
Basic unit	Function / procedure	Object (instance of a class)
Focus	Steps of the algorithm (*how* to do it)	Data and its behaviour (*what* things are)
Design approach	Top-down: break the task into sub-tasks	Bottom-up: identify the objects, then their interactions
Data and functions	Kept separate; data passed between functions or held globally	Bundled together inside objects (**encapsulation**)
Data security	Global data can be changed by any function	`private` members hide data (**information hiding**)
Reuse	Copy or call library functions	**Inheritance**: new classes extend existing ones
Flexibility	A change to a data structure affects every function using it	**Polymorphism**: one interface, many behaviours (virtual functions)
Best suited to	Small to medium, calculation-heavy programs; embedded code	Large, complex systems that change over time; GUIs; simulations
Overheads	Simple, fast, small memory use	More design effort; slightly more memory and run-time overhead

Conclusion [4]: neither is better in every case. Structured programming is ideal for clear, calculation-driven tasks (an SCS 1101 tariff or statistics program); OOP pays off as programs grow, because encapsulation, inheritance and polymorphism keep large systems modular and easy to extend. Modern practice combines them: OOP for the overall architecture, structured code inside each method.

Engineering lens: a structural-analysis program written in a **structured** style has functions such as `readLoads()`, `solveStiffness()` and `printResults()`. In an **object-oriented** style it has `Beam`, `Column` and `Node` classes, each knowing its own properties and how to compute its own stiffness. Adding a new member type then means adding one class, not editing every function: exactly the modular benefit of polymorphism.

Examiner's tip: for a 25-mark essay, a table of 8–10 contrasts plus a short paragraph each for definitions, similarities and conclusion is the most efficient way to reach full marks.

Sources for this book

- [MIT-2] MIT 6.006 Lecture 19 — recursive vs memoized Fibonacci (Q3b).
- [UCLA-3] UCLA CS 31 syllabus — intro programming taught in C++ (context for this paper's language).
- [NUST-1] SCS 1101 course outline — 40% CA / 60% exam weighting used in Q5b.
- Textbook knowledge (tier 4): structured constructs, walk-throughs, loops, arrays, C++ classes, constructors, inline and virtual functions, inheritance types, structured vs OOP.

Source register

The full register for the solution-book set. Sources marked ✓ are cited in this book; the rest (MIT, UCLA and Wits material used across the set) are further reading on the same topics.

Key	Source	Institution	Used for	In this book
NUST-1	SCS 1101 course outline (uploaded)	NUST	Syllabus, assessment, C as the course language	✓
MIT-1	6.087 Practical Programming in C	MIT	C syntax, pointers, structs, files, coding style	Further reading
MIT-2	6.006 Lecture 19: Dynamic Programming I (Fibonacci)	MIT	Why naive recursive Fibonacci is slow; memoization	✓
MIT-3	Whirlwind Computer, 1944–59 (IEEE Milestone)	MIT	Magnetic-core memory, real-time computing	Further reading
MIT-4	Wang et al., *Undefined Behavior: What Happened to My Code?*	MIT CSAIL	Run-time and undefined-behaviour errors	Further reading
UCLA-1	First internet message, 29 Oct 1969	UCLA Samueli	History of networks and the internet	Further reading
UCLA-2	Birthplace of the Internet (IEEE Milestone)	UCLA	ARPANET, packet switching, first node	Further reading
UCLA-3	CS 31 Introduction to Computer Science I, syllabus	UCLA	Intro-programming scope; portability across compilers	✓
WITS-1	Sithole (Wits) & Ruhode, *Cloud Computing Adoption: Opportunities and Challenges for SMMEs in South Africa*, arXiv:2108.10079	Wits	Cloud benefits and barriers in Southern Africa	Further reading

Key	Source	Institution	Used for	In this book
WITS-2	*Leadership and the adoption of cloud computing for enterprises in Johannesburg* (2022)	Wits	Drivers and barriers of cloud adoption	Further reading
STD-1	NIST SP 800-145, *The NIST Definition of Cloud Computing*	NIST	SaaS, PaaS, IaaS; 5 characteristics; 4 deployment models	Further reading
RES-1	Bakasa & Pekane, *Decision Criteria for Cloud Adoption by Large Enterprises in South Africa*, arXiv:2108.09757	UCT	Security and data sovereignty as selection criteria	Further reading
RES-2	Fu et al., *Error Log Analysis in C Programming Language Courses* (ICCE 2015)	Kyushu University	The errors novice C students actually make	Further reading